

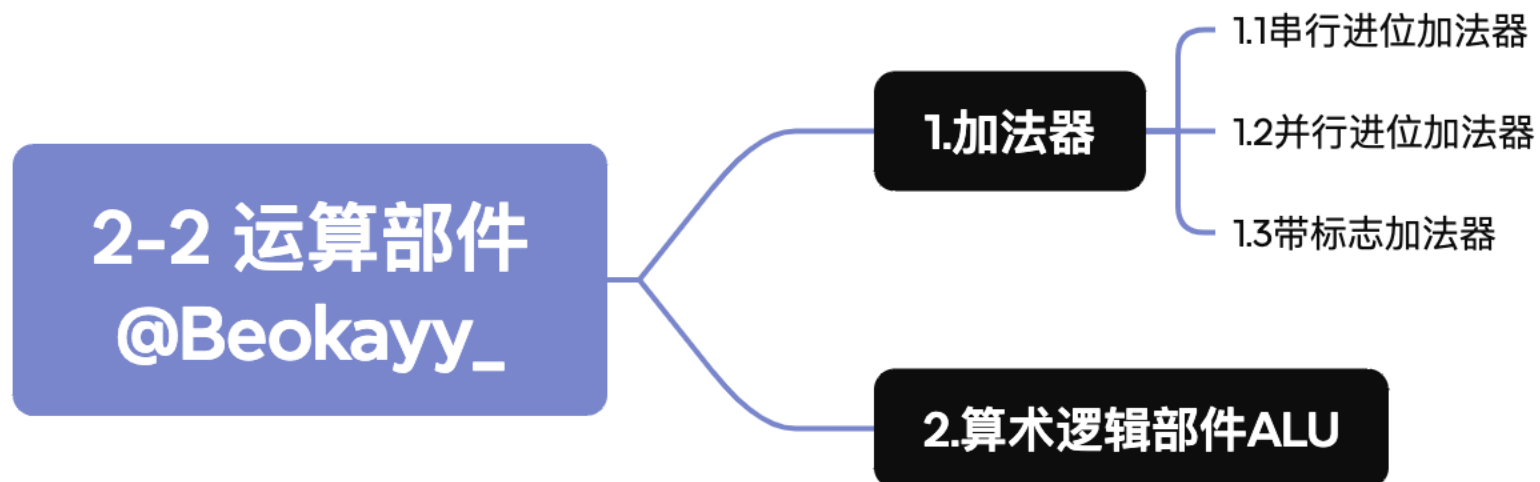
bok25

**基
础**

班、计算机组成原理



运算部件





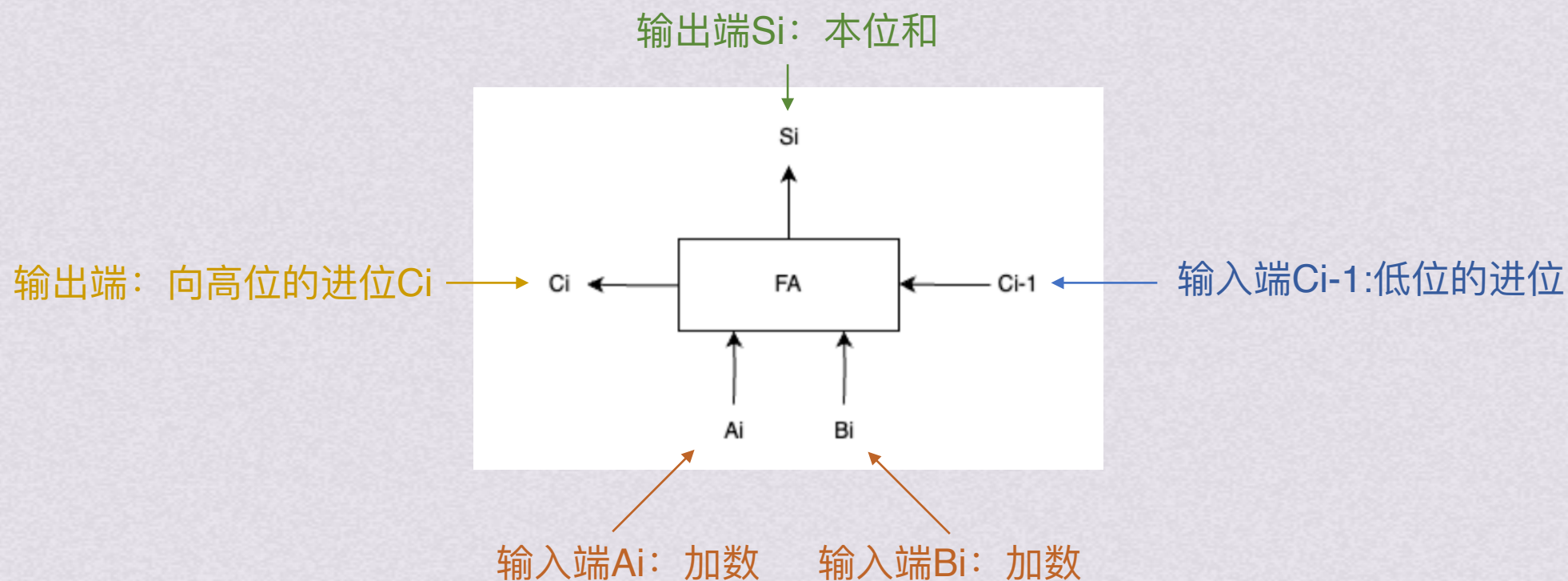
1.加法器



运算部件

1. 加法器

全加器



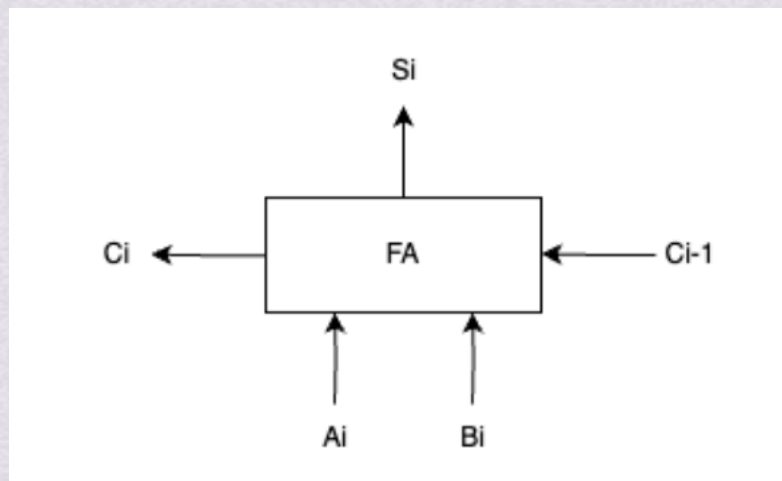


运算部件

1. 加法器

全加器

$$\begin{array}{r} + 1 \\ 0 \\ \hline \end{array}$$



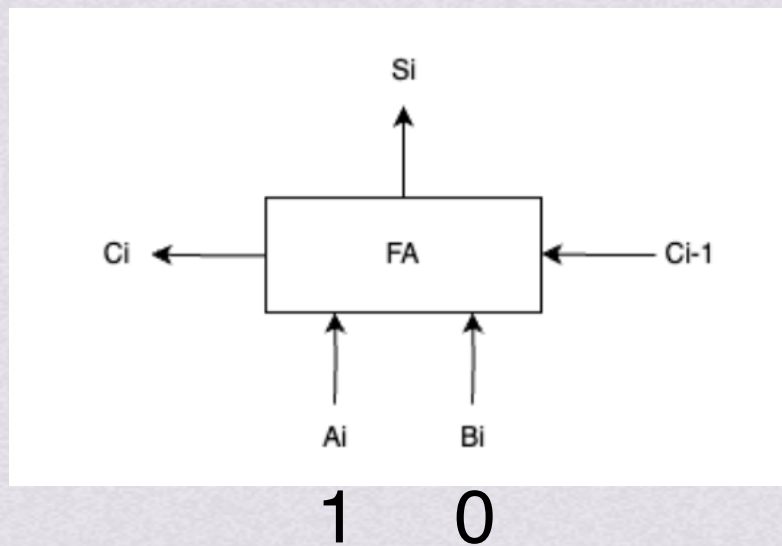


运算部件

1. 加法器

全加器

+



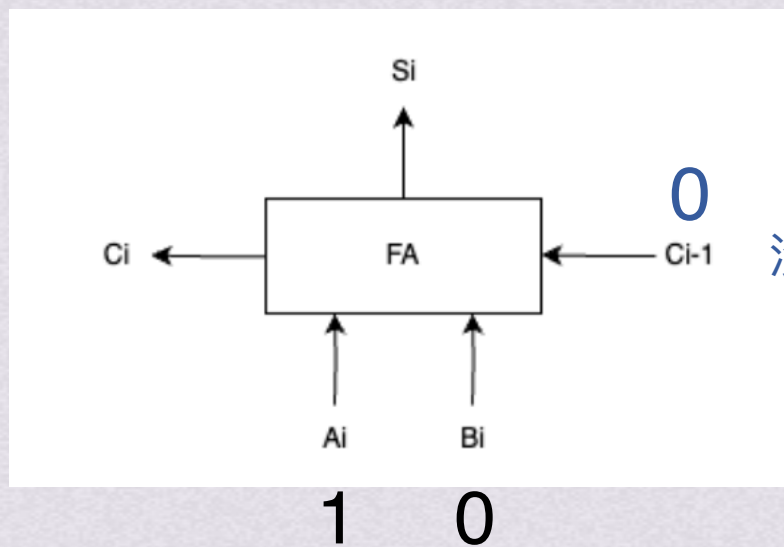


运算部件

1. 加法器

全加器

+

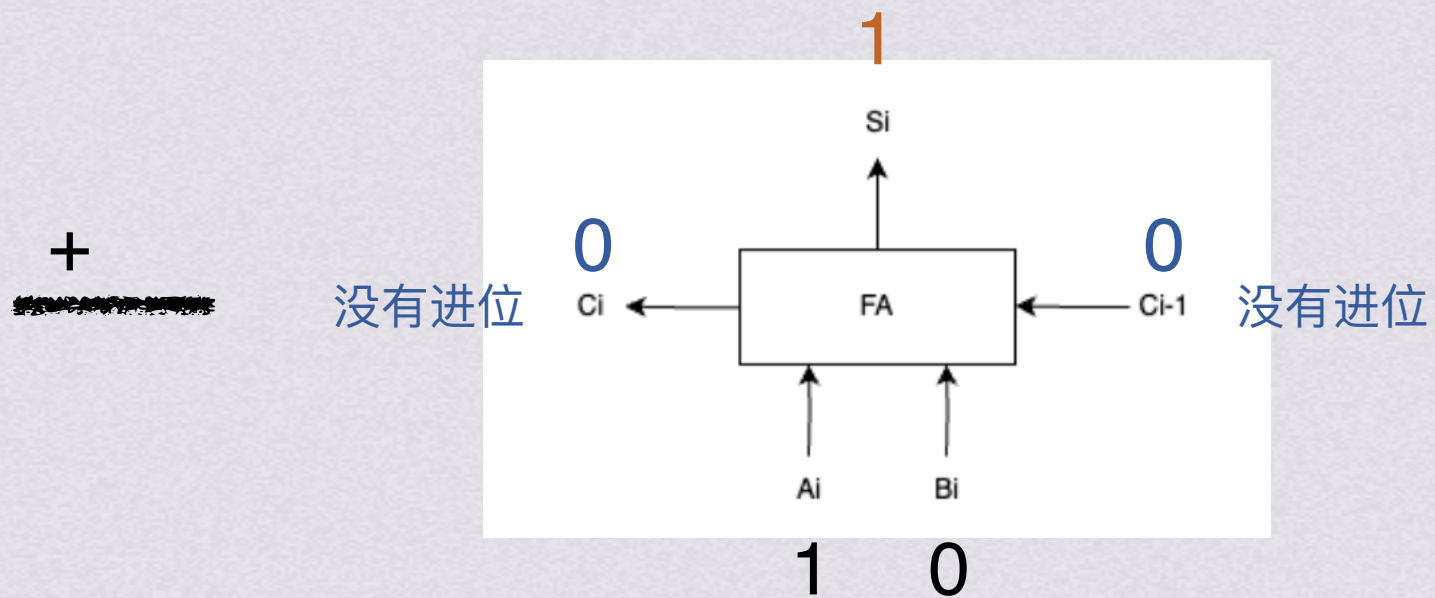




运算部件

1. 加法器

全加器





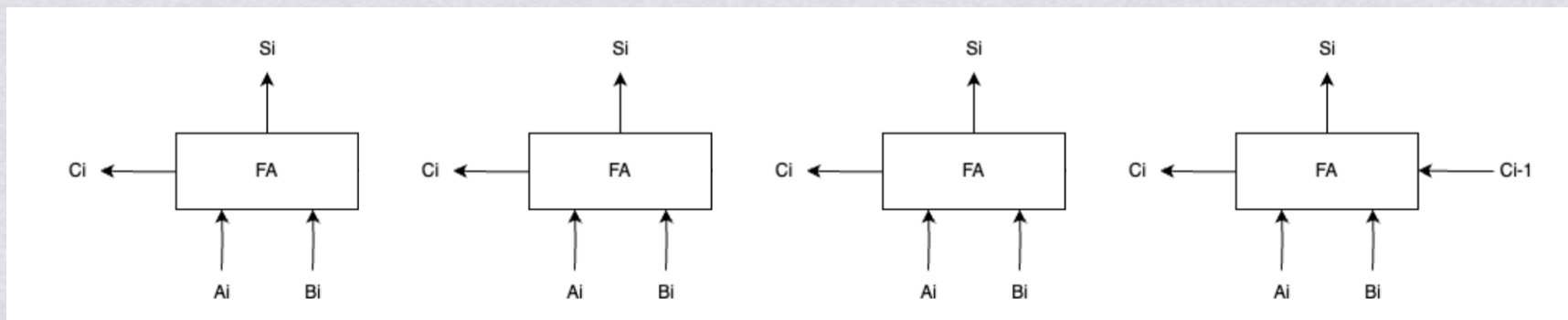
运算部件

1. 加法器

1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



1101
+ 0101

~~11010101010101010101010101010101~~

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)



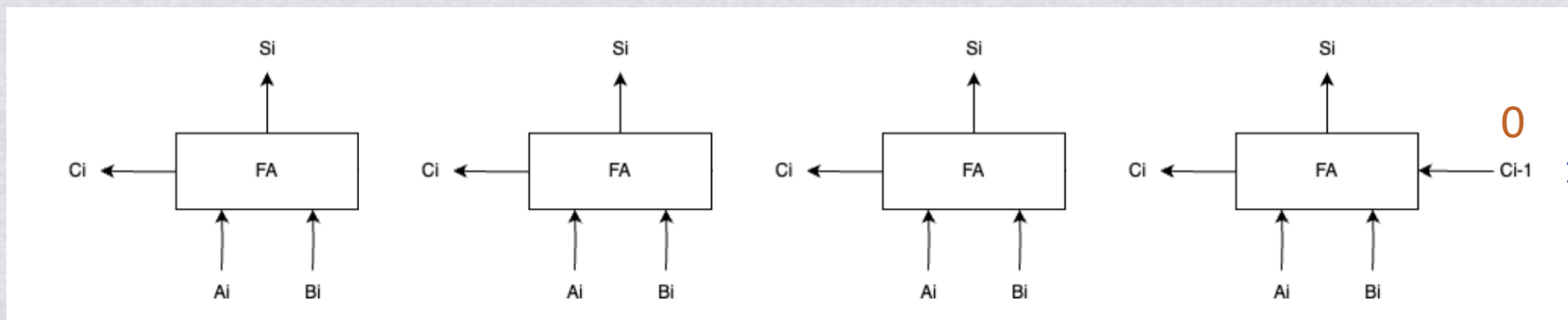
运算部件

1. 加法器

1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



1101
+ 0101

~~11010101~~

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)



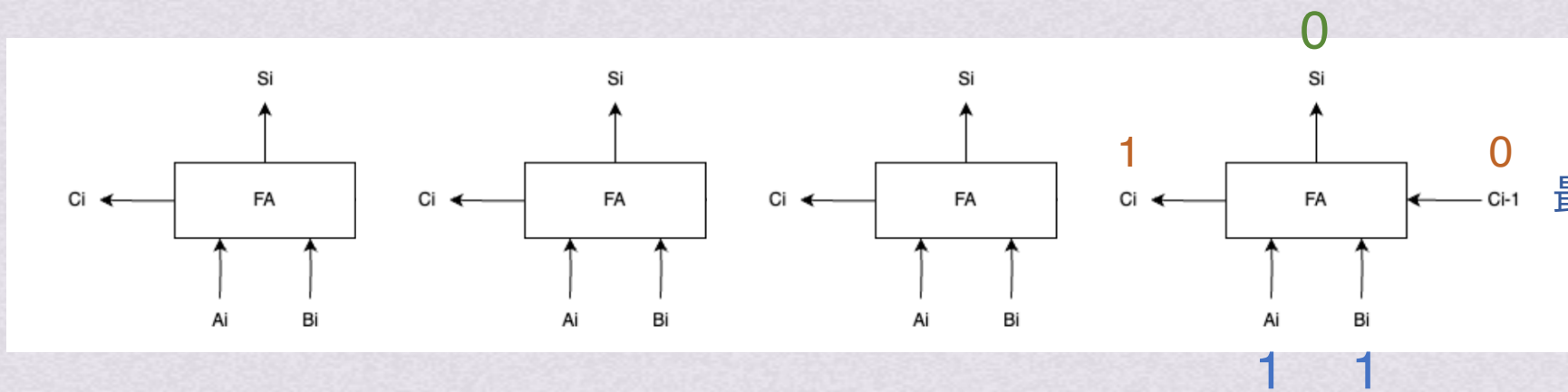
运算部件

1. 加法器

1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



最低位没有进位

$$\begin{array}{r} 1101 \\ + 0101 \\ \hline \end{array}$$

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)

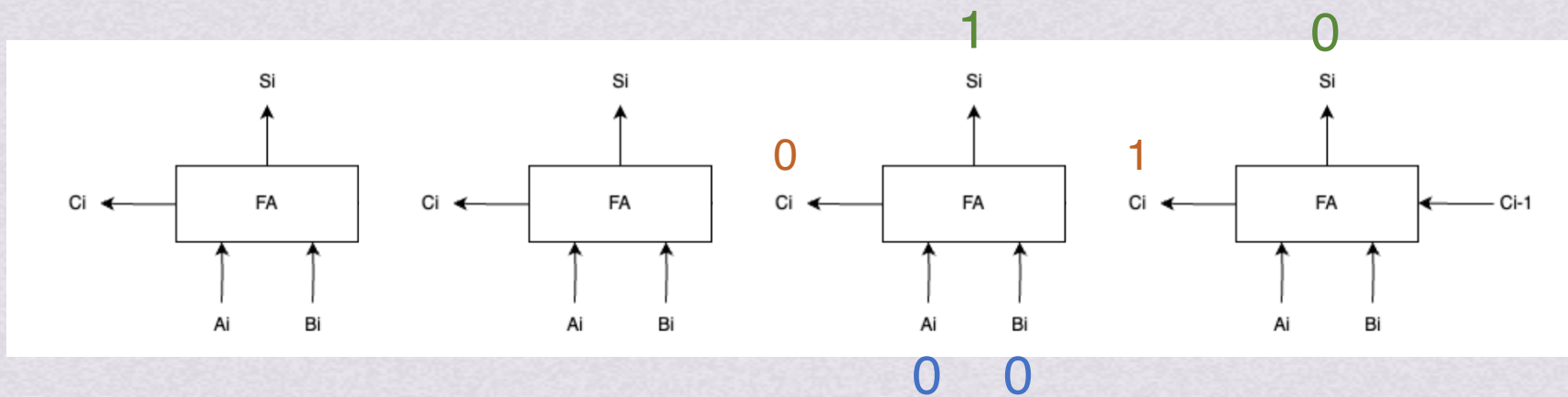


运算部件

1. 加法器 1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



1101
+ 0101

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)

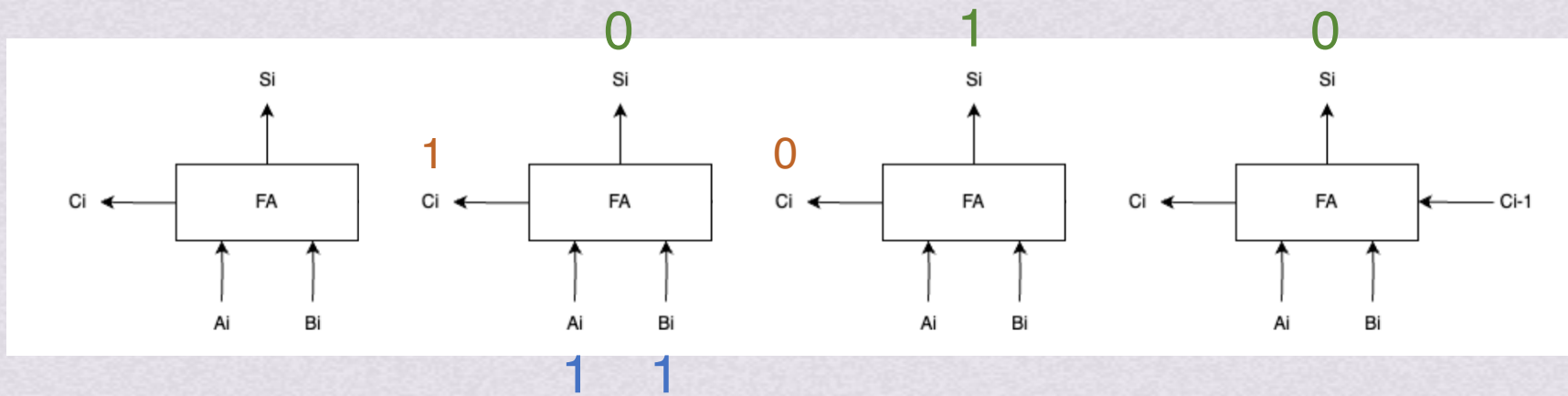


运算部件

1. 加法器 1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



1101
+ 0101

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)

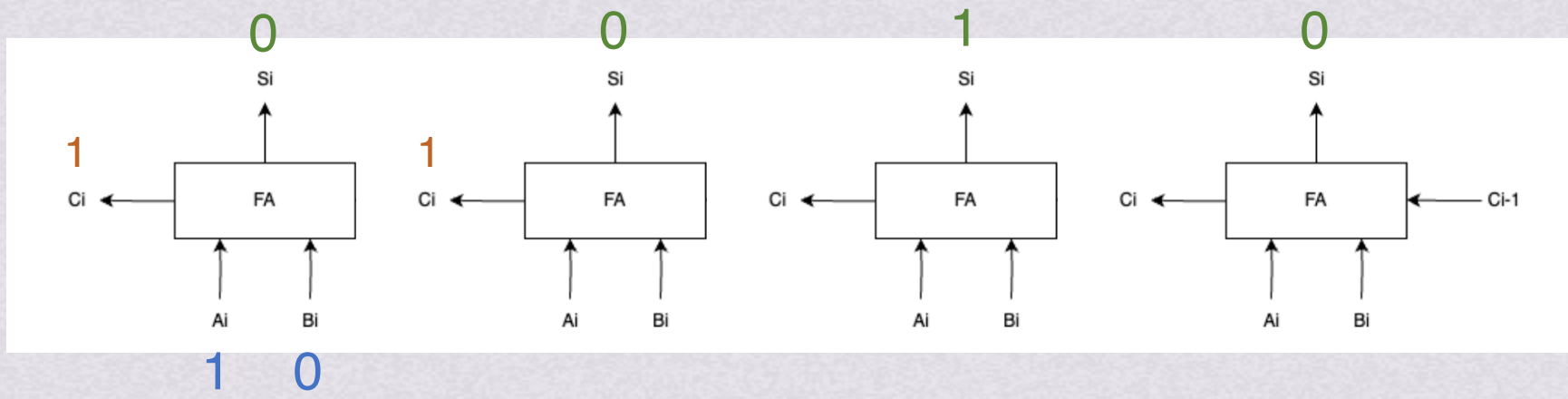


运算部件

1. 加法器 1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



1101
+ 0101

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

(实际上就是2进制加法)



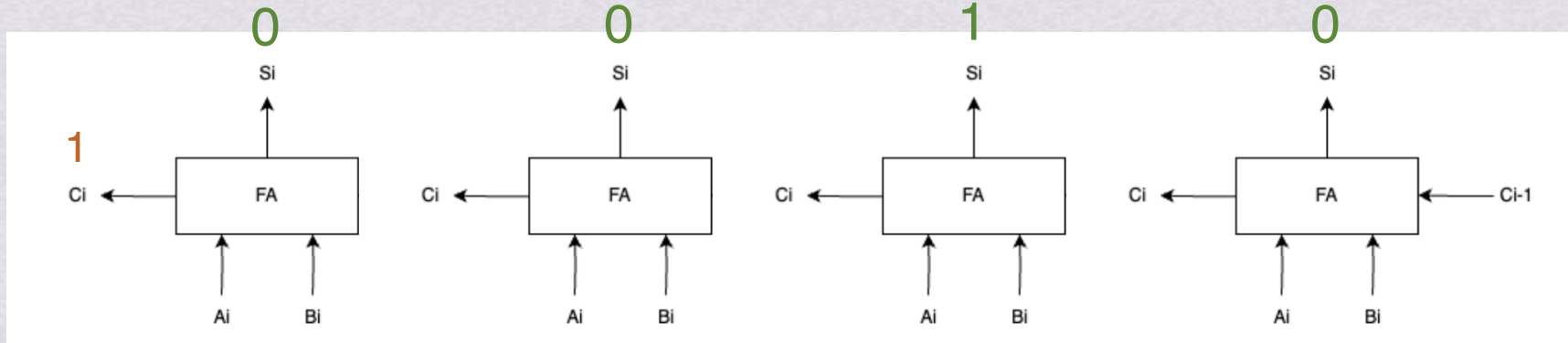
运算部件

1. 加法器

1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器



$$\begin{array}{r} 1101 \\ + 0101 \\ \hline \end{array}$$



运算部件

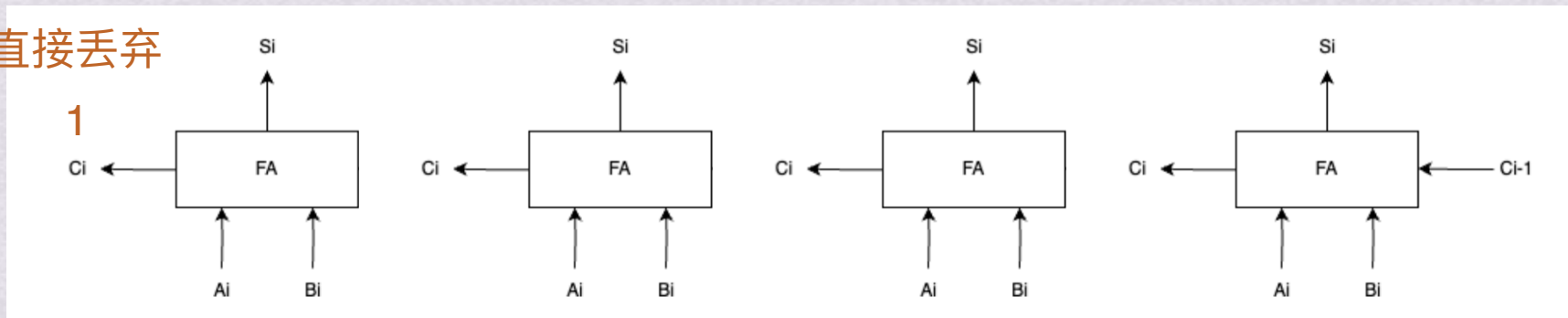
1. 加法器

1.1 串行进位加法器

串行进位加法器

将n个一位全加器相连即可得到n位串行进位加法器

最高位进位直接丢弃



$$\begin{array}{r} 1101 \\ + 0101 \\ \hline 0010 \end{array}$$

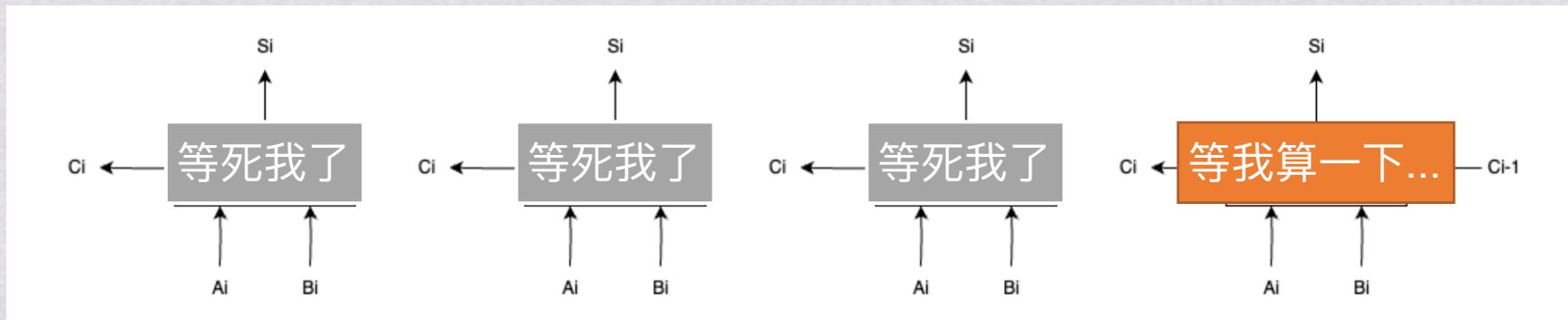


运算部件

1.加法器 1.1 串行进位加法器

缺陷

低位运算产生进位所需的时间将影响高位运算的时间，位数越多延迟时间越长



$$\begin{array}{r} 1101 \\ + 0101 \\ \hline 0 \end{array}$$



运算部件

1. 加法器 1.2 并行进位加法器

并行进位加法器

相互间的进位没有依赖关系

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

P_i G_i

$$C_i = P_i C_{i-1} + G_i$$



运算部件

1. 加法器 1.2 并行进位加法器

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

P_i G_i

$$C_i = P_i C_{i-1} + G_i$$

并行进位加法器

相互间的进位没有依赖关系

$$C_1 = G_1 + P_1 C_0$$

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 (G_1 + P_1 C_0) = G_2 + P_2 G_1 + P_1 P_2 C_0$$

$$C_3 = G_3 + P_3 C_2 = G_3 + P_3 (G_2 + P_2 G_1 + P_1 P_2 C_0) = G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 C_0$$

...

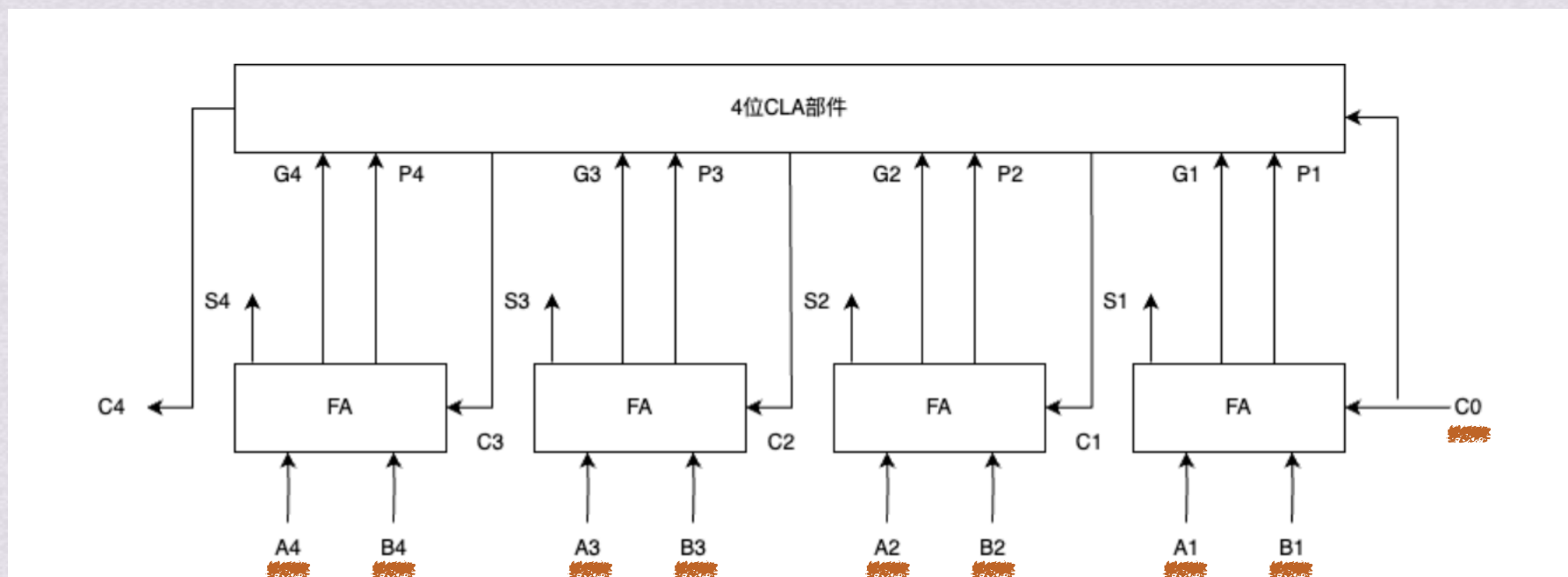


运算部件

1.加法器 1.2 并行进位加法器

并行进位加法器

相互间的进位没有依赖关系



只要让A1~A4， B1~B4以及C0同时到达，就可以几乎同时形成C1~C4以及每位和Si，得到结果

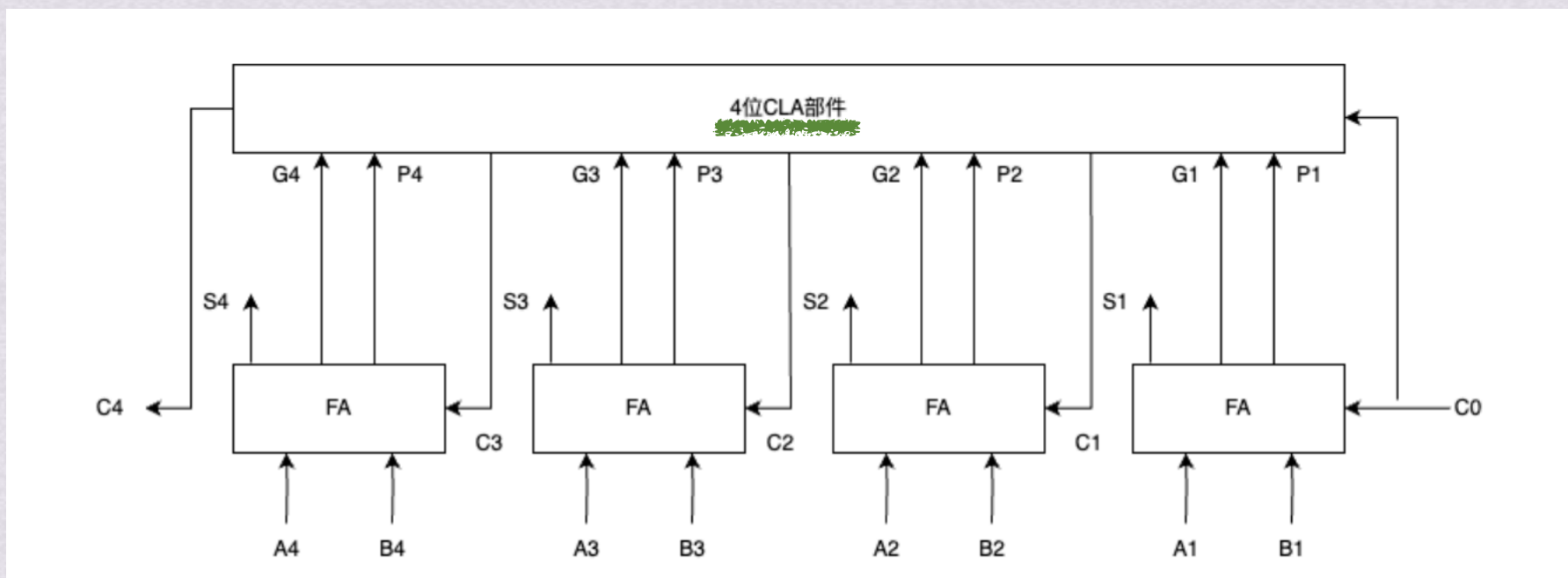


运算部件

1.加法器 1.2 并行进位加法器

并行进位加法器

相互间的进位没有依赖关系



CLA部件可以实现计算逻辑

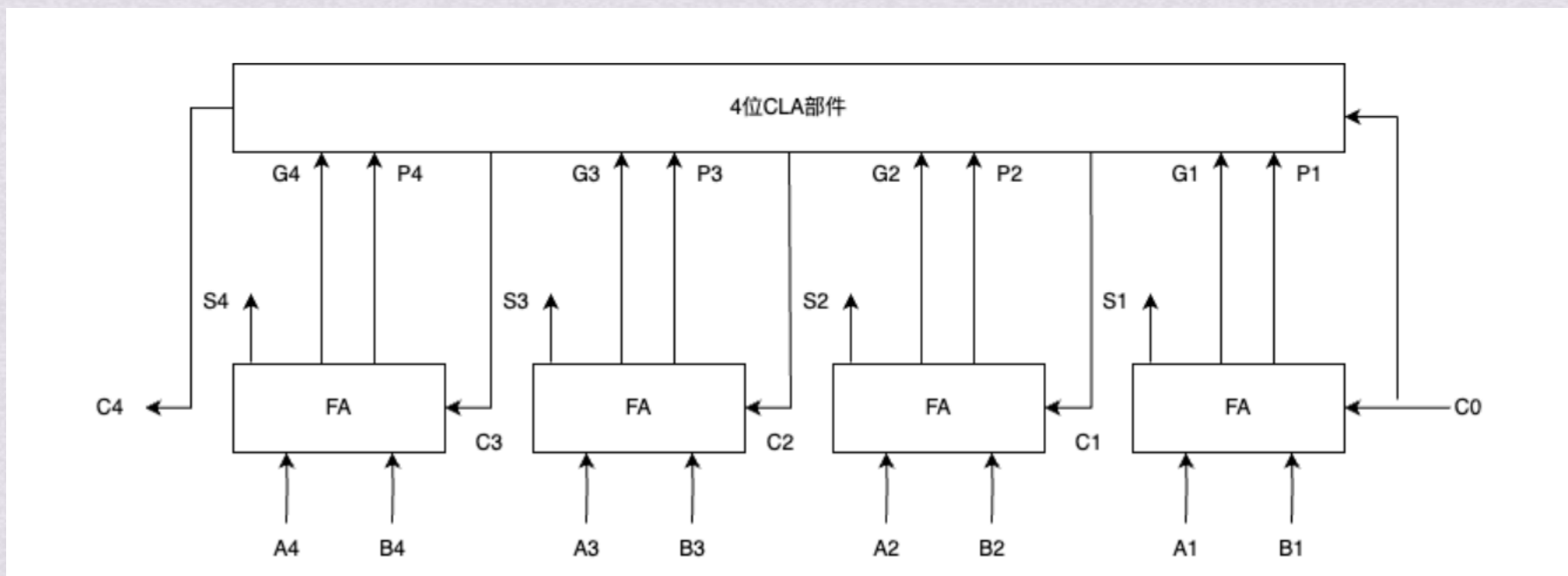


运算部件

1.加法器 1.2 并行进位加法器

并行进位加法器

相互间的进位没有依赖关系



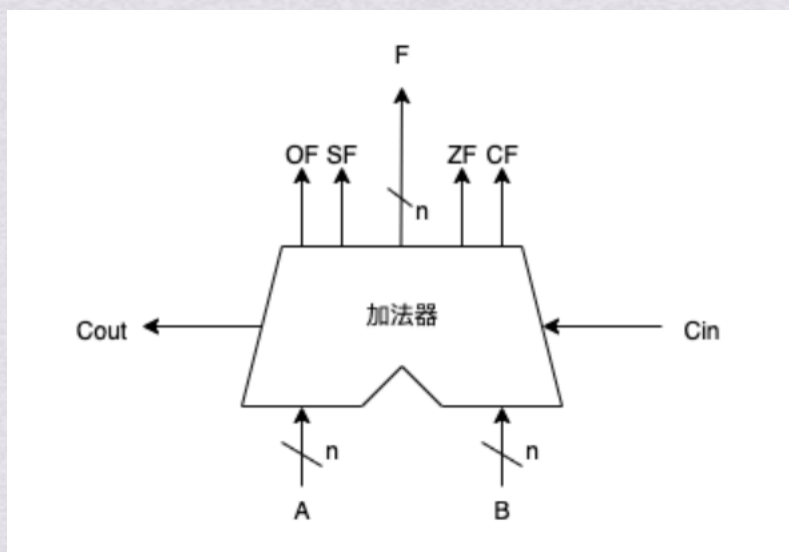
并行加法器的计算速度与位数无关，但随着位数的增加， C_i 的逻辑表达式会非常复杂，导致电路也十分复杂，难以实现



运算部件

1. 加法器 1.3 带标志加法器

带标志加法器



$$OF = C_n \oplus C_{n-1}$$

OF为溢出标志， C_n 为最高位进位， C_{n-1} 为次高位进位

仅对带符号数运算有效



运算部件

1. 加法器 1.3 带标志加法器

带标志加法器

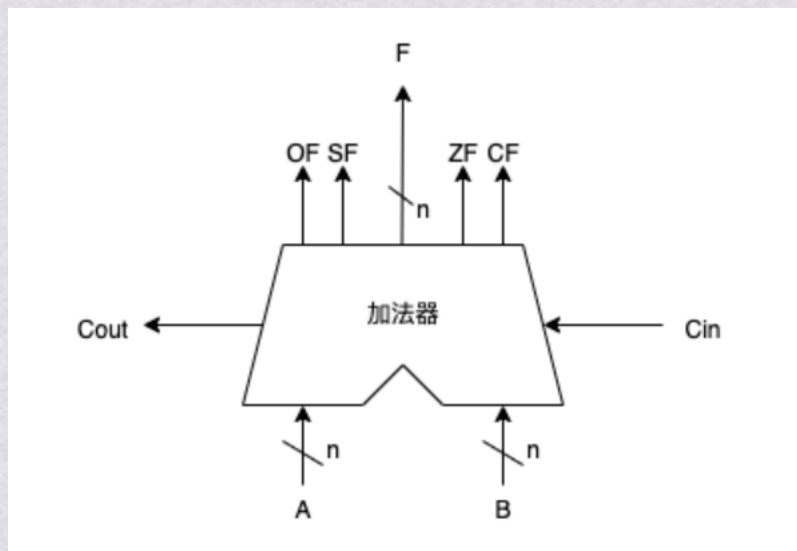
$$OF = C_n \oplus C_{n-1}$$

OF为溢出标志， C_n 为最高位进位， C_{n-1} 为次高位进位

【例1】 $x=0011$ ， $y=0101$ ，求 $x+y$

$$OF = 1 \oplus 0 = 1$$

发生溢出，结果错误

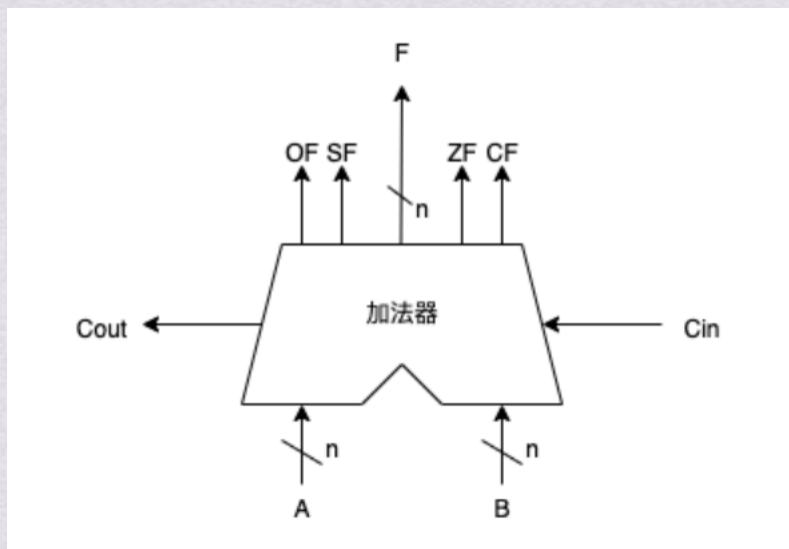




运算部件

1. 加法器 1.3 带标志加法器

带标志加法器



SF即为结果的符号

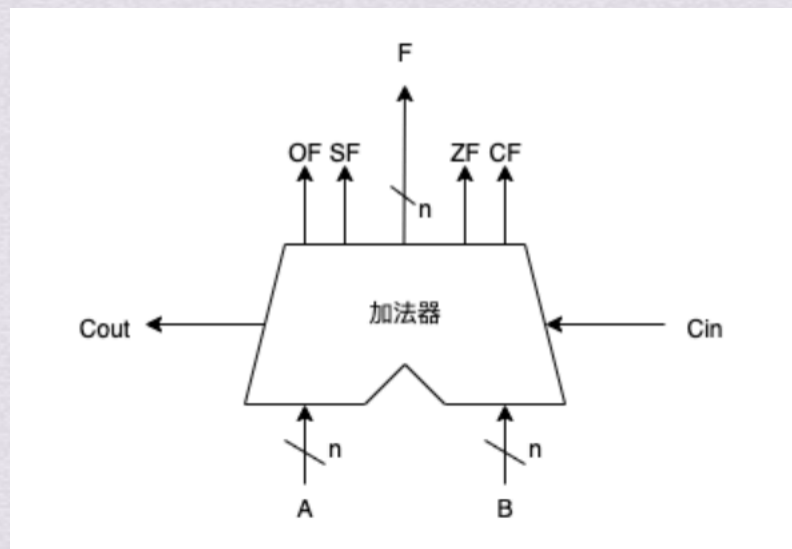
SF=F，若SF=1，结果为负；SF=0，结果为正
仅对带符号数运算有效



运算部件

1. 加法器 1.3 带标志加法器

带标志加法器



ZF为零标志，代表结果是否为0

ZF=1代表结果为0

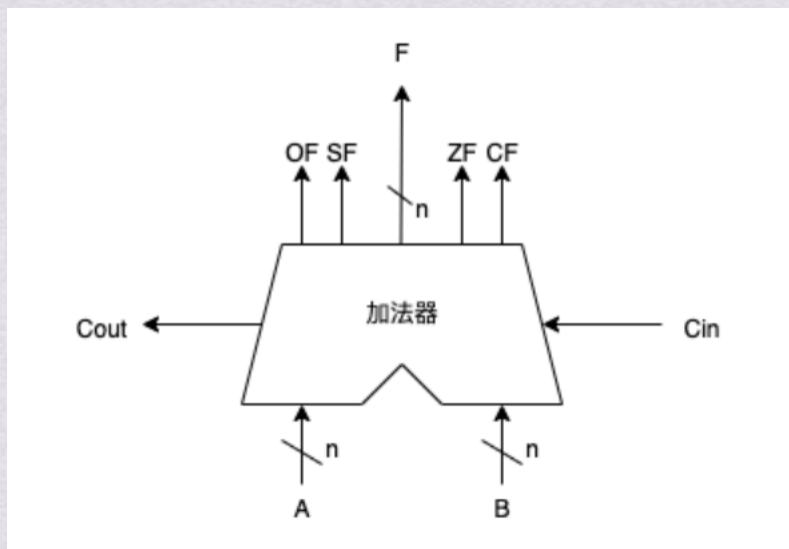
对有符号数和无符号数都有效



运算部件

1. 加法器 1.3 带标志加法器

带标志加法器



$$CF = Cout \oplus Cin$$

CF为进位/借位符号，Cout为最高位进位，Cin为最低位进位

仅对无符号数运算有效



运算部件

1.加法器 1.3 带标志加法器

带标志加法器

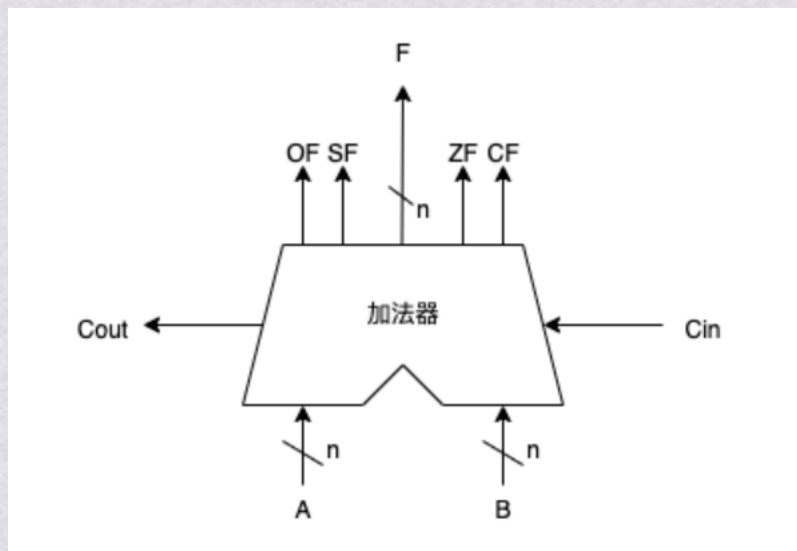
$$CF = Cout \oplus Cin$$

CF为进位/借位符号，Cout为最高位进位，Cin为最低位进位

【例2】 $x=1011$ ， $y=0101$ ，求 $x+y$

$$CF = 1 \oplus 0 = 1$$

发生进位，结果错误





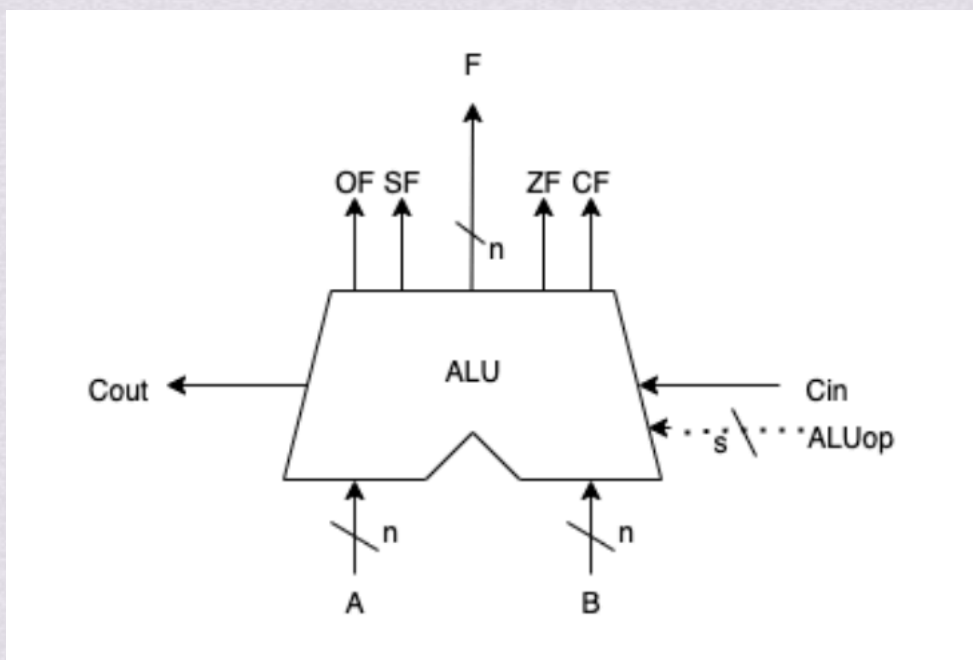
2.算术逻辑单元ALU



运算部件

2. 算数逻辑单元ALU

算术逻辑单元ALU



A和B是操作数输入端

Cin是进位输入端

ALUop是操作控制端，用于决定ALU执行的功能

Cout是进位输出端

F是计算结果输出端